Jenkins — Revision Notes / Cheatsheet

Core Concepts

- **Controller (Master)**: schedules builds, dispatches to agents, serves the web UI, stores config.
- **Agent (Node)**: executes the actual build steps. Can be static (permanent) or dynamic (Docker/K8s/cloud).
- **Executor**: a slot on a node that can run one build at a time.
- **Job/Project**: a configured unit of work (Freestyle, Pipeline, Multibranch Pipeline, etc.).
- **Build**: one execution of a job.
- **Workspace**: the directory on the agent where a job's files live during a build.

Job Types

```
| Type | Use case |

|------|----------|

| Freestyle project | Simple, UI-configured jobs — good for learning, not for complex logic |

| Pipeline | Jenkinsfile-based, code-as-config, supports Declarative & Scripted syntax |

| Multibranch Pipeline | Auto-discovers branches/PRs in a repo, each gets its own pipeline run |

| Folder | Organizational grouping of jobs |
```

Declarative Pipeline Skeleton

```
pipeline {
    agent any
    environment {
        APP_ENV = 'staging'
    }
    stages {
        stage('Checkout') {
            steps { checkout scm }
        }
        stage('Build') {
            steps { sh 'make build' }
        }
        stage('Test') {
            steps { sh 'make test' }
        }
        stage('Deploy') {
            when { branch 'main' }
            steps { sh 'make deploy' }
        }
    }
    post {
```

```
        always { echo 'Pipeline finished' }
        failure { echo 'Pipeline failed' }
    }
}
```

Key Plugins

- **Git / GitHub / GitHub Branch Source** — SCM integration
- **Pipeline** (and Pipeline: Stage View) — Jenkinsfile support + visualization
- **Credentials Binding** — inject secrets into pipelines safely
- **Docker Pipeline** — build/run containers as build steps or agents
- **Blue Ocean** — modern pipeline visualization UI
- **Role-based Authorization Strategy** — fine-grained access control

Credential Types

- Username/password
- SSH username with private key
- Secret text (API tokens)
- Secret file
- Certificate

Always reference credentials by ID, never hardcode secrets in a Jenkinsfile:

```
withCredentials([usernamePassword(credentialsId: 'docker-hub', usernameVariable: 'USER', passwordVariable: '
    sh 'docker login -u $USER -p $PASS'
}
```

Triggers

- **Poll SCM**: `H/5 * * * *` — cron-style polling
- **GitHub webhook**: push-based, near-instant, preferred over polling
- **Build periodically**: pure cron, independent of SCM changes
- **Build after other projects**: upstream/downstream chaining

Useful `sh` / Groovy Snippets

```
// Parallel stages
stage('Test') {
    parallel {
        stage('Unit') { steps { sh 'npm run test:unit' } }
        stage('Lint') { steps { sh 'npm run lint' } }
    }
}
// Retry a flaky step
```

```
retry(3) { sh './flaky-script.sh' }

// Timeout a stage
timeout(time: 10, unit: 'MINUTES') { sh './long-task.sh' }
```

Backup Essentials (JENKINS_HOME)

- `config.xml` — global config
- `jobs/` — all job definitions and build history
- `secrets/` — encryption keys (critical — back these up!)
- `plugins/` — installed plugins

See Also

- [commands/jenkins.md](commands/jenkins-md) — CLI & general commands
- [jenkins-pipeline.md](commands/jenkins-pipeline.md) — pipeline-specific syntax
- troubleshooting/jenkins.md

Final Jenkins Quiz — Days 11-15 Comprehensive

Section A: Fundamentals (Day 11)

Define controller, agent, and executor in one sentence each.

What's the risk of running production builds directly on the controller?

Section B: Jobs & Pipelines (Day 12)

Compare Declarative and Scripted pipeline syntax.

Write a minimal Declarative pipeline with a parameter and a conditional deploy stage (pseudocode is fine).

Section C: GitHub Integration (Day 13)

Explain the full webhook-to-build flow.

What's the difference between building a PR's head commit vs the merge result?

Section D: Jenkins + Docker (Day 14)

Compare Docker-outside-of-Docker vs Docker-in-Docker.

What tagging strategy would you use for CI-built images, and why?

Section E: Production Jenkins (Day 15)

List three components of a production hardening checklist for Jenkins.

How would you structure credentials and RBAC for a 5-team organization sharing one Jenkins instance?

Scenario Question

Your Multibranch Pipeline builds are queued but never start, even though the controller shows low CPU/memory usage. Walk through your triage steps.

<details><summary>Scenario answer notes</summary>

Check (in order): 1) job's node/label restriction vs available agent labels, 2) agent connectivity (`Manage Jenkins → Nodes` — is it marked offline?), 3) executor count on matching agents (all busy?), 4) `disableConcurrentBuilds()` blocking a new build behind a stuck prior run, 5) Docker/Kubernetes cloud agent provisioning failures in the logs if using dynamic agents.

</details>

Self-Assessment

- [ ] I can explain Jenkins architecture without notes
- [ ] I can write a Declarative pipeline with parameters, parallel stages, and post actions from scratch
- [ ] I can set up GitHub webhook + Multibranch Pipeline integration end to end
- [ ] I can build, tag, push, and deploy a Docker image from a pipeline
- [ ] I can describe a production-hardening checklist (RBAC, backups, scaling, shared libraries)